

贪心

每一步都要有理由地做出选择

Rosaya

Oasis

目录

1. 贪心思路	2	3.4 习题	23
1.1 定义	3	4. 反悔贪心	24
1.2 区间贪心	6	4.1 思想	25
1.3 习题	10	4.2 区间覆盖限制	26
2. 简单贪心策略	11	4.3 后悔药模型	29
2.1 排序后扫描	12	4.4 习题	30
2.2 构造性贪心	13		
2.3 双指针贪心	15		
2.4 习题	16		
3. 相邻交换	17		
3.1 基本方法	18		
3.2 国王游戏	19		
3.3 公式化比较	21		

目录

1. 贪心思路	2	3.4 习题	23
1.1 定义	3	4. 反悔贪心	24
1.2 区间贪心	6	4.1 思想	25
1.3 习题	10	4.2 区间覆盖限制	26
2. 简单贪心策略	11	4.3 后悔药模型	29
2.1 排序后扫描	12	4.4 习题	30
2.2 构造性贪心	13		
2.3 双指针贪心	15		
2.4 习题	16		
3. 相邻交换	17		
3.1 基本方法	18		
3.2 国王游戏	19		
3.3 公式化比较	21		

1.1 定义

1.1 定义

贪心算法是在每一步都选择当前看来最优的决策，并希望这些局部最优决策能组成全局最优解的算法。

1.1 定义

贪心算法是在每一步都选择当前看来最优的决策，并希望这些局部最优决策能组成全局最优解的算法。

贪心最容易写，也最容易写错。一个贪心策略不是“看起来对”就可以用，而是必须回答两个问题：

- 为什么当前选择不会让最优解变差？
- 为什么做完当前选择后，剩下的问题仍然是同类问题？

1.1 定义

贪心算法是在每一步都选择当前看来最优的决策，并希望这些局部最优决策能组成全局最优解的算法。

贪心最容易写，也最容易写错。一个贪心策略不是“看起来对”就可以用，而是必须回答两个问题：

- **为什么当前选择不会让最优解变差？**
- **为什么做完当前选择后，剩下的问题仍然是同类问题？**

常见证明方式：

- **交换论证**：把任意最优解调整成包含当前贪心选择的最优解。
- **归纳证明**：证明第一步贪心选择正确，剩余部分递归成立。
- **反证法**：假设最优解不按贪心策略做，推出可以变得更优或不变。

- **不变量**：证明算法过程中维护了某个最优性条件。

1.1 定义

1.1 定义

贪心的基本分析流程：

1. 找到一个可比较的局部选择，例如最早结束、最小代价、最大收益、最高优先级。
2. 尝试证明存在一个最优解包含这个选择。
3. 将这个选择固定，观察剩余问题是否仍可用同样方式解决。
4. 检查边界情况和相等时的排序规则。

1.1 定义

贪心的基本分析流程：

1. 找到一个可比较的局部选择，例如最早结束、最小代价、最大收益、最高优先级。
2. 尝试证明存在一个最优解包含这个选择。
3. 将这个选择固定，观察剩余问题是否仍可用同样方式解决。
4. 检查边界情况和相等时的排序规则。

重点：贪心不是枚举少了的 DP，而是证明某些状态永远不需要考虑。

1.2 区间贪心

1.2 区间贪心

- P1803 凌乱的 yyy / 线段覆盖

给定 n 个比赛的开始时间和结束时间，每场比赛只能完整参加，求最多能参加多少场。

$1 \leq n \leq 10^6$ 。

1.2 区间贪心

- P1803 凌乱的 yyy / 线段覆盖

给定 n 个比赛的开始时间和结束时间，每场比赛只能完整参加，求最多能参加多少场。

$1 \leq n \leq 10^6$ 。

策略：按结束时间从小到大排序，能选就选。

1.2 区间贪心

- P1803 凌乱的 yyy / 线段覆盖

给定 n 个比赛的开始时间和结束时间，每场比赛只能完整参加，求最多能参加多少场。

$1 \leq n \leq 10^6$ 。

策略：按结束时间从小到大排序，能选就选。

为什么不是按开始时间、持续时间或者空闲时间排序？

因为我们选完当前比赛后，唯一影响后续选择的是**当前结束时间**。结束越早，留给后面比赛的时间越多。

1.2 区间贪心

- P1803 凌乱的 yyy / 线段覆盖

给定 n 个比赛的开始时间和结束时间，每场比赛只能完整参加，求最多能参加多少场。

$1 \leq n \leq 10^6$ 。

策略：按结束时间从小到大排序，能选就选。

为什么不是按开始时间、持续时间或者空闲时间排序？

因为我们选完当前比赛后，唯一影响后续选择的是**当前结束时间**。结束越早，留给后面比赛的时间越多。

交换证明：设最优解第一场选择了 x ，而所有比赛中结束最早且可选的是 g 。因为 g 的结束时间不晚于 x ，将 x 替换为 g 后，后续所有原本能接在 x 后面的比赛仍能接在 g 后面，所以答案不变。于是存在一个最优解第一步选择 g 。

剩下的比赛仍然是同样的问题，归纳即可。

1.2 区间贪心

1.2 区间贪心

```
1  struct Seg {  
2      int l, r;  
3  };  
4  
5  sort(a + 1, a + n + 1, [](Seg x, Seg y) {  
6      if (x.r != y.r) return x.r < y.r;  
7      return x.l < y.l;  
8  });  
9  
10 int last = -1, ans = 0;
```



```
11  for (int i = 1; i <= n; i++) {  
12      if (a[i].l >= last) {  
13          ans++;  
14          last = a[i].r;  
15      }  
16  }
```

1.2 区间贪心

```
1  struct Seg {  
2      int l, r;  
3  };  
4  
5  sort(a + 1, a + n + 1, [](Seg x, Seg y) {  
6      if (x.r != y.r) return x.r < y.r;  
7      return x.l < y.l;  
8  });  
9  
10 int last = -1, ans = 0;
```



```
11  for (int i = 1; i <= n; i++) {  
12      if (a[i].l >= last) {  
13          ans++;  
14          last = a[i].r;  
15      }  
16  }
```

这里的相等条件要看题目定义：如果两段允许端点相接，使用 `a[i].l >= last`；如果要求严格不重叠，则改为 `a[i].l > last`。

1.3 习题

- P1803 凌乱的 yyy / 线段覆盖
- P1094 [NOIP2007 普及组] 纪念品分组
- P2887 [USACO07NOV] Sunscreen G
- P1250 种树

目录

1. 贪心思路	2	3.4 习题	23
1.1 定义	3	4. 反悔贪心	24
1.2 区间贪心	6	4.1 思想	25
1.3 习题	10	4.2 区间覆盖限制	26
2. 简单贪心策略	11	4.3 后悔药模型	29
2.1 排序后扫描	12	4.4 习题	30
2.2 构造性贪心	13		
2.3 双指针贪心	15		
2.4 习题	16		
3. 相邻交换	17		
3.1 基本方法	18		
3.2 国王游戏	19		
3.3 公式化比较	21		

2.1 排序后扫描

2.1 排序后扫描

很多贪心题的第一步是排序。排序的意义不是“让数据有序”，而是**制造一个可以从左到右确定的决策顺序**。

2.1 排序后扫描

很多贪心题的第一步是排序。排序的意义不是“让数据有序”，而是**制造一个可以从左到右确定的决策顺序**。

- **P1223 排队接水**

有 n 个人排队接水，第 i 个人需要 t_i 的时间。求一种排队顺序，使平均等待时间最小。

2.1 排序后扫描

很多贪心题的第一步是排序。排序的意义不是“让数据有序”，而是**制造一个可以从左到右确定的决策顺序**。

- **P1223 排队接水**

有 n 个人排队接水，第 i 个人需要 t_i 的时间。求一种排队顺序，使平均等待时间最小。

策略：按接水时间从小到大排序。

2.1 排序后扫描

很多贪心题的第一步是排序。排序的意义不是“让数据有序”，而是**制造一个可以从左到右确定的决策顺序**。

- **P1223 排队接水**

有 n 个人排队接水，第 i 个人需要 t_i 的时间。求一种排队顺序，使平均等待时间最小。

策略：按接水时间从小到大排序。

相邻交换证明：考虑相邻两个人 i, j ，且当前只关心这两人的相对顺序。若 i 在前，二者对总等待时间的贡献是 t_i ；若 j 在前，贡献是 t_j 。为了让贡献更小，应让较短者在前。

如果序列中存在逆序对 $t_i > t_j$ ，交换它们不会让答案变差。不断交换后得到非降序排列，所以该顺序最优。

2.2 构造性贪心

2.2 构造性贪心

- P1106 删数问题

给定一个十进制数，删除其中 k 位，使剩余数字按原顺序组成的新数最小。

2.2 构造性贪心

- P1106 删数问题

给定一个十进制数，删除其中 k 位，使剩余数字按原顺序组成的新数最小。

策略：从左到右维护一个单调不降的栈，遇到更小的数字时，尽量删除前面较大的数字。

2.2 构造性贪心

- P1106 删数问题

给定一个十进制数，删除其中 k 位，使剩余数字按原顺序组成的新数最小。

策略：从左到右维护一个单调不降的栈，遇到更小的数字时，尽量删除前面较大的数字。

为什么优先改前面的位？

十进制比较中，越靠前的位权重越大。若前一位能变小，那么后面怎么变化都无法抵消它带来的优势。

2.2 构造性贪心

• P1106 删数问题

给定一个十进制数，删除其中 k 位，使剩余数字按原顺序组成的新数最小。

策略：从左到右维护一个单调不降的栈，遇到更小的数字时，尽量删除前面较大的数字。

为什么优先改前面的位？

十进制比较中，越靠前的位权重越大。若前一位能变小，那么后面怎么变化都无法抵消它带来的优势。

```
1  string s, st;
```

```
2  int k;
```

```
3
```



```
4  for (char c : s) {  
5      while (!st.empty() && k > 0 && st.back() > c) {  
6          st.pop_back();  
7          k--;  
8      }  
9      st.push_back(c);  
10 }  
11  
12 while (k > 0) st.pop_back(), k--;
```

2.2 构造性贪心

• P1106 删数问题

给定一个十进制数，删除其中 k 位，使剩余数字按原顺序组成的新数最小。

策略：从左到右维护一个单调不降的栈，遇到更小的数字时，尽量删除前面较大的数字。

为什么优先改前面的位？

十进制比较中，越靠前的位权重越大。若前一位能变小，那么后面怎么变化都无法抵消它带来的优势。

```
1  string s, st;
```

```
2  int k;
```

```
3
```




```
4  for (char c : s) {  
5      while (!st.empty() && k > 0 && st.back() > c) {  
6          st.pop_back();  
7          k--;  
8      }  
9      st.push_back(c);  
10 }  
11  
12 while (k > 0) st.pop_back(), k--;
```

最后要处理前导零和删空的情况。

重点：这类题的贪心本质是“越靠前越重要”。

2.3 双指针贪心

2.3 双指针贪心

- P1094 [NOIP2007 普及组] 纪念品分组

有 n 件纪念品，每组最多两件，且一组重量和不能超过 w 。求最少分成多少组。

2.3 双指针贪心

- P1094 [NOIP2007 普及组] 纪念品分组

有 n 件纪念品，每组最多两件，且一组重量和不能超过 w 。求最少分成多少组。

策略：排序后用双指针。每次让最重的物品单独成组，若它能和当前最轻的物品同组，就把最轻的也带上。

2.3 双指针贪心

- P1094 [NOIP2007 普及组] 纪念品分组

有 n 件纪念品，每组最多两件，且一组重量和不能超过 w 。求最少分成多少组。

策略：排序后用双指针。每次让最重的物品单独成组，若它能和当前最轻的物品同组，就把最轻的也带上。

证明：最重的物品一定要进入某一组。

若它能和最轻的同组，那么把最轻的与它配对不会影响其他物品可行性，因为最轻的最容易被替换。

若它不能和最轻的同组，那么它不能和任何其他物品同组，只能单独一组。

2.3 双指针贪心

- P1094 [NOIP2007 普及组] 纪念品分组

有 n 件纪念品，每组最多两件，且一组重量和不能超过 w 。求最少分成多少组。

策略：排序后用双指针。每次让最重的物品单独成组，若它能和当前最轻的物品同组，就把最轻的也带上。

证明：最重的物品一定要进入某一组。

若它能和最轻的同组，那么把最轻的与它配对不会影响其他物品可行性，因为最轻的最容易被替换。

若它不能和最轻的同组，那么它不能和任何其他物品同组，只能单独一组。

重点：双指针贪心常常把“最难安排”的元素先固定。

2.4 习题

- P1223 排队接水
- P1106 删数问题
- P1094 [NOIP2007 普及组] 纪念品分组
- P2240 【深基 12.例 1】部分背包问题
- P4995 跳跳！

目录

1. 贪心思路	2	3.4 习题	23
1.1 定义	3	4. 反悔贪心	24
1.2 区间贪心	6	4.1 思想	25
1.3 习题	10	4.2 区间覆盖限制	26
2. 简单贪心策略	11	4.3 后悔药模型	29
2.1 排序后扫描	12	4.4 习题	30
2.2 构造性贪心	13		
2.3 双指针贪心	15		
2.4 习题	16		
3. 相邻交换	17		
3.1 基本方法	18		
3.2 国王游戏	19		
3.3 公式化比较	21		

3.1 基本方法

3.1 基本方法

相邻交换是排序贪心中最常见的证明工具。

3.1 基本方法

相邻交换是排序贪心中最常见的证明工具。

假设答案由某个排列决定。我们只比较相邻两个元素 a, b 的顺序：

- 若 a, b 放成 a, b 比放成 b, a 更优，则最优排列中不应该出现 b, a 这样的逆序。
- 由此得到一个比较规则，再按这个规则排序。

3.1 基本方法

相邻交换是排序贪心中最常见的证明工具。

假设答案由某个排列决定。我们只比较相邻两个元素 a, b 的顺序：

- 若 a, b 放成 a, b 比放成 b, a 更优，则最优排列中不应该出现 b, a 这样的逆序。
- 由此得到一个比较规则，再按这个规则排序。

重点：相邻交换不是为了推出冒泡排序，而是为了推出排序比较函数。

3.2 国王游戏

3.2 国王游戏

- P1080 [NOIP2012 提高组] 国王游戏

国王和 n 个大臣都有左手数 l_i 和右手数 r_i 。队伍中第 i 个大臣得到的金币数为他前面所有人的左手数乘积除以自己的右手数，求一种排列，使所有大臣获得金币数的最大值最小。

$1 \leq n \leq 1000$ ，数值可能很大。

3.2 国王游戏

• P1080 [NOIP2012 提高组] 国王游戏

国王和 n 个大臣都有左手数 l_i 和右手数 r_i 。队伍中第 i 个大臣得到的金币数为他前面所有人的左手数乘积除以自己的右手数，求一种排列，使所有大臣获得金币数的最大值最小。

$1 \leq n \leq 1000$ ，数值可能很大。

考虑相邻两个大臣 a, b ，设前面的乘积为 S 。

若顺序为 a, b ，最大贡献为：

$$\max \left\{ \frac{S}{r_a}, S \frac{l_a}{r_b} \right\}$$

若顺序为 b, a ，最大贡献为：

$$\max\left\{\frac{S}{r_b}, S\frac{l_b}{r_a}\right\}$$

3.2 国王游戏

• P1080 [NOIP2012 提高组] 国王游戏

国王和 n 个大臣都有左手数 l_i 和右手数 r_i 。队伍中第 i 个大臣得到的金币数为他前面所有人的左手数乘积除以自己的右手数，求一种排列，使所有大臣获得金币数的最大值最小。

$1 \leq n \leq 1000$ ，数值可能很大。

考虑相邻两个大臣 a, b ，设前面的乘积为 S 。

若顺序为 a, b ，最大贡献为：

$$\max \left\{ \frac{S}{r_a}, S \frac{l_a}{r_b} \right\}$$

若顺序为 b, a ，最大贡献为：

$$\max\left\{\frac{S}{r_b}, S\frac{l_b}{r_a}\right\}$$

去掉共同的 S 后，可以证明若 $l_a r_a < l_b r_b$ ，则 a 应排在 b 前面。

所以按 $l_i r_i$ 从小到大排序。

3.2 国王游戏

• P1080 [NOIP2012 提高组] 国王游戏

国王和 n 个大臣都有左手数 l_i 和右手数 r_i 。队伍中第 i 个大臣得到的金币数为他前面所有人的左手数乘积除以自己的右手数，求一种排列，使所有大臣获得金币数的最大值最小。

$1 \leq n \leq 1000$ ，数值可能很大。

考虑相邻两个大臣 a, b ，设前面的乘积为 S 。

若顺序为 a, b ，最大贡献为：

$$\max \left\{ \frac{S}{r_a}, S \frac{l_a}{r_b} \right\}$$

若顺序为 b, a ，最大贡献为：

$$\max\left\{\frac{S}{r_b}, S\frac{l_b}{r_a}\right\}$$

去掉共同的 S 后，可以证明若 $l_a r_a < l_b r_b$ ，则 a 应排在 b 前面。

所以按 $l_i r_i$ 从小到大排序。

注意这题需要高精度。贪心得到顺序，计算答案时仍要维护大整数。

3.3 公式化比较

3.3 公式化比较

- P2123 皇后游戏

有若干任务，每个任务在两台机器上依次加工，任务 i 在第一台机器耗时 a_i ，在第二台机器耗时 b_i 。求最小完工时间。

3.3 公式化比较

- P2123 皇后游戏

有若干任务，每个任务在两台机器上依次加工，任务 i 在第一台机器耗时 a_i ，在第二台机器耗时 b_i 。求最小完工时间。

考虑两个相邻任务 i, j 的顺序。

若 i 在前，局部影响可写成：

$$\max\{a_i + a_j + b_j, a_i + b_i + b_j\}$$

若 j 在前，局部影响可写成：

$$\max\{a_j + a_i + b_i, a_j + b_j + b_i\}$$

3.3 公式化比较

- P2123 皇后游戏

有若干任务，每个任务在两台机器上依次加工，任务 i 在第一台机器耗时 a_i ，在第二台机器耗时 b_i 。求最小完工时间。

考虑两个相邻任务 i, j 的顺序。

若 i 在前，局部影响可写成：

$$\max\{a_i + a_j + b_j, a_i + b_i + b_j\}$$

若 j 在前，局部影响可写成：

$$\max\{a_j + a_i + b_i, a_j + b_j + b_i\}$$

整理后可以得到 Johnson 法则：

- 若 $\min(a_i, b_j) < \min(a_j, b_i)$, 则 i 应排在 j 前。

3.3 公式化比较

- P2123 皇后游戏

有若干任务，每个任务在两台机器上依次加工，任务 i 在第一台机器耗时 a_i ，在第二台机器耗时 b_i 。求最小完工时间。

考虑两个相邻任务 i, j 的顺序。

若 i 在前，局部影响可写成：

$$\max\{a_i + a_j + b_j, a_i + b_i + b_j\}$$

若 j 在前，局部影响可写成：

$$\max\{a_j + a_i + b_i, a_j + b_j + b_i\}$$

整理后可以得到 Johnson 法则：

- 若 $\min(a_i, b_j) < \min(a_j, b_i)$, 则 i 应排在 j 前。

这类题的难点不在写代码, 而在把相邻交换的局部代价写对。

3.4 习题

- P1080 [NOIP2012 提高组] 国王游戏
- P2123 皇后游戏
- P1842 [USACO05NOV] 奶牛玩杂技
- P1248 加工生产调度
- CF1539D PriceFixed

目录

1. 贪心思路	2	3.4 习题	23
1.1 定义	3	4. 反悔贪心	24
1.2 区间贪心	6	4.1 思想	25
1.3 习题	10	4.2 区间覆盖限制	26
2. 简单贪心策略	11	4.3 后悔药模型	29
2.1 排序后扫描	12	4.4 习题	30
2.2 构造性贪心	13		
2.3 双指针贪心	15		
2.4 习题	16		
3. 相邻交换	17		
3.1 基本方法	18		
3.2 国王游戏	19		
3.3 公式化比较	21		

4.1 思想

4.1 思想

有些问题中，当前看起来最优的选择可能会被后面的选择推翻。

如果我们直接放弃贪心，就会变成复杂的 DP；但如果能用一个数据结构记录“已经做过的最差选择”，就可以在后面反悔。

4.1 思想

有些问题中，当前看起来最优的选择可能会被后面的选择推翻。

如果我们直接放弃贪心，就会变成复杂的 DP；但如果能用一个数据结构记录“已经做过的最差选择”，就可以在后面反悔。

反悔贪心的一般形式：

1. 按某个顺序扫描事件。
2. 遇到可选物品先选上。
3. 若约束被破坏，就撤销已经选择的某个最差物品。
4. 用堆维护可以被撤销的选择。

4.1 思想

有些问题中，当前看起来最优的选择可能会被后面的选择推翻。

如果我们直接放弃贪心，就会变成复杂的 DP；但如果能用一个数据结构记录“已经做过的最差选择”，就可以在后面反悔。

反悔贪心的一般形式：

1. 按某个顺序扫描事件。
2. 遇到可选物品先选上。
3. 若约束被破坏，就撤销已经选择的某个最差物品。
4. 用堆维护可以被撤销的选择。

重点：反悔贪心并不是真的回溯，而是维护“当前前缀内的最优集合”。

4.2 区间覆盖限制

4.2 区间覆盖限制

- **P2949 [USACO09OPEN] Work Scheduling G**

有 n 个工作，每个工作有截止时间 d_i 和收益 p_i ，每个工作耗时 1。求最大收益。

4.2 区间覆盖限制

- **P2949 [USACO09OPEN] Work Scheduling G**

有 n 个工作，每个工作有截止时间 d_i 和收益 p_i ，每个工作耗时 1。求最大收益。

策略：按截止时间从小到大排序，依次尝试选择每个工作。

用小根堆维护已经选择的工作的收益。

若当前已经选择的工作数量超过当前截止时间，就删除收益最小的工作。

4.2 区间覆盖限制

- P2949 [USACO09OPEN] Work Scheduling G

有 n 个工作，每个工作有截止时间 d_i 和收益 p_i ，每个工作耗时 1。求最大收益。

策略：按截止时间从小到大排序，依次尝试选择每个工作。

用小根堆维护已经选择的工作的收益。

若当前已经选择的工作数量超过当前截止时间，就删除收益最小的工作。

为什么正确？

扫描到截止时间 d 时，我们只需要在所有截止时间不超过 d 的工作中选出至多 d 个。为了最大化收益，显然应该保留收益最大的 d 个。小根堆正是在维护这个集合。

4.2 区间覆盖限制

4.2 区间覆盖限制

```
1  sort(a + 1, a + n + 1, [](Job x, Job y) {  
2      return x.d < y.d;  
3  });  
4  
5  priority_queue<int, vector<int>, greater<int>> q;  
6  long long ans = 0;  
7  
8  for (int i = 1; i <= n; i++) {  
9      q.push(a[i].p);  
10     ans += a[i].p;
```



```
11     if ((int)q.size() > a[i].d) {  
12         ans -= q.top();  
13         q.pop();  
14     }  
15 }
```

4.3 后悔药模型

4.3 后悔药模型

- P1484 种树

从一排位置中选择若干个不相邻的位置，使权值和最大。

4.3 后悔药模型

- P1484 种树

从一排位置中选择若干个不相邻的位置，使权值和最大。

一个经典做法是每次选择当前最大权值的位置，然后把它左右相邻的位置删除。为了允许之后反悔，把当前位置的权值改成：

$$a_l + a_r - a_i$$

再放回堆中。

4.3 后悔药模型

- P1484 种树

从一排位置中选择若干个不相邻的位置，使权值和最大。

一个经典做法是每次选择当前最大权值的位置，然后把它左右相邻的位置删除。为了允许之后反悔，把当前位置的权值改成：

$$a_l + a_r - a_i$$

再放回堆中。

含义：如果之后重新选择这个位置，相当于撤销之前选 i ，改为选择 l, r 。

这种做法把“选了以后不能选相邻”转化成了“每次选择一个增量，并允许用负增量反悔”。

4.3 后悔药模型

- P1484 种树

从一排位置中选择若干个不相邻的位置，使权值和最大。

一个经典做法是每次选择当前最大权值的位置，然后把它左右相邻的位置删除。为了允许之后反悔，把当前位置的权值改成：

$$a_l + a_r - a_i$$

再放回堆中。

含义：如果之后重新选择这个位置，相当于撤销之前选 i ，改为选择 l, r 。

这种做法把“选了以后不能选相邻”转化成了“每次选择一个增量，并允许用负增量反悔”。

这类反悔贪心通常需要链表维护左右邻居，并用堆维护当前最大增量。

4.4 习题

- P2949 [USACO09OPEN] Work Scheduling G
- P1484 种树
- P3045 [USACO12FEB] Cow Coupons G
- P3620 [APIO/CTSC 2007] 数据备份
- CF865D Buy Low Sell High

Thanks!